Manhattan Distance

```
import heapq

GOAL = (1, 2, 3,
        8, 0, 4,
        7, 6, 5)


def manhattan_distance(state):
    distance = 0

    for i in range(9):
        if state[i] != 0:
            current_row, current_col = divmod(i, 3)
            goal_pos = GOAL.index(state[i])
            goal_row, goal_col = divmod(goal_pos, 3)

            distance += abs(current_row - goal_row)
            distance += abs(current_col - goal_col)

    return distance


def get_neighbors(state):
    neighbors = []

    zero = state.index(0)
    row, col = divmod(zero, 3)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_pos = new_row * 3 + new_col

            new_state = list(state)
            new_state[zero], new_state[new_pos] = \
                new_state[new_pos], new_state[zero]

            neighbors.append(tuple(new_state))
```

```
    return neighbors


def a_star(start):
    h = manhattan_distance(start)

    pq = [(h, 0, start, [start])]
    visited = {start: 0}

    while pq:
        f, g, current, path = heapq.heappop(pq)

        if current == GOAL:
            return path

        for neighbor in get_neighbors(current):
            new_g = g + 1

            if neighbor not in visited or new_g < visited[neighbor]:
                visited[neighbor] = new_g

                h = manhattan_distance(neighbor)
                new_f = new_g + h

                heapq.heappush(
                    pq,
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    return None


def print_board(state):
    for i in range(0, 9, 3):
        print(*state[i:i + 3])
    print("-" * 9)


def get_input():
    while True:
        try:
            values = tuple(map(int, input("Enter 9 numbers: ").split()))
```

```
            if len(values) != 9:
                print("Enter exactly 9 numbers.")
                continue

            if sorted(values) != list(range(9)):
                print("Enter numbers 0 to 8 exactly once.")
                continue

            return values

        except ValueError:
            print("Enter numbers only.")


start = get_input()

print("\nInitial State:")
print_board(start)

print("Goal State:")
print_board(GOAL)

solution = a_star(start)

if solution:
    print("Minimum steps:", len(solution) - 1)
    print()

    for step, state in enumerate(solution):
        print("Step", step)
        print_board(state)
else:
    print("No solution found.")
```

```
Enter 9 numbers: 8 1 3 7 2 4 6 0 5


Initial State:
8 1 3
7 2 4
6 0 5
---------
Goal State:
1 2 3
8 0 4
7 6 5
---------
Minimum steps: 5

Step 0
8 1 3
7 2 4
6 0 5
---------
Step 1
8 1 3
7 2 4
0 6 5
---------
Step 2
8 1 3
0 2 4
7 6 5
---------
Step 3
0 1 3
8 2 4
7 6 5
---------
Step 4
1 0 3
8 2 4
7 6 5
---------
Step 5
1 2 3
8 0 4
7 6 5
---------
```